FAKE EXAM — RANK 03 PYTHON

LEVEL 1

Assignment name : mirror_case

Expected files : mirror_case.py
Allowed functions : len, .upper, .lower

Write a function:

```
def mirror_case(s):
```

This function receives a string.

Your goal is to alternate the case of alphabetic characters.

The first alphabetic character must become uppercase. The second alphabetic character must become lowercase. The third alphabetic character must become uppercase again, and so on.

Non alphabetic characters must stay unchanged and must NOT affect the alternation order.

You must return the transformed string.

Example:
```
mirror_case("hello world!")
```

Output:
```
"HeLlO WoRlD!"
```

Assignment name : rotate_train

Expected files : rotate_train.py
Allowed functions : len

Write a function:

```
def rotate_train(lst, n):
```

This function receives a list and an integer.

The function must rotate the list to the right n times.

The last elements become the first elements.

If n is bigger than the list size, the function must still work correctly.

You must return the rotated list.

Example:
`rotate_train([1,2,3,4,5], 2)`

Output:
`[4,5,1,2,3]`

LEVEL 2

Assignment name : hidden_word

Expected files : hidden_word.py
Allowed functions : len

Write a function:

```
def hidden_word(small, big):
```

The function receives two strings.

Return True if all characters of the first string appear inside the second string in the same order.

Characters do not need to be consecutive.

Return False otherwise.

Example:
```
hidden_word("abc", "a1b2c3")
```

Output:
```
True
```

Assignment name : smart_sort

Expected files : smart_sort.py
Allowed functions : sorted, len

Write a function that sorts a list of strings using multiple rules.

The list must be sorted:
- first by string length
- then alphabetically
- then by number of vowels if still equal

Strings with fewer vowels must come first.

You must return the new sorted list.

Example:
```
smart_sort(["banana", "cat", "apple", "dog"])
```

LEVEL 3

Assignment name : brackets

Expected files : brackets.py
Allowed functions : len

Write a function that checks if every bracket is correctly closed.

You must support:
- ()
- []
- {}

Return True if the expression is valid. Return False otherwise.

A closing bracket must always match the latest opened bracket.

Example:
```
brackets("([]{})")
```

Output:
```
True
```

Assignment name : base_convert

Expected files : base_convert.py
Allowed functions : len

Write a function that converts a decimal number into another base.

The function must support bases from 2 to 16.

Bases bigger than 10 must use uppercase letters:
- A for 10
- B for 11
- etc...

You must return the converted value as a string.

Example:
```
base_convert(255, 16)
```

Output:
```
"FF"
```

LEVEL 4

Assignment name : matrix_reverse

Expected files : matrix_reverse.py
Allowed functions : len

Write a function that reverses a matrix both vertically and horizontally.

The last row becomes the first row. The last column becomes the first column.

You must return the transformed matrix.

Example:
`matrix_reverse([[1,2],[3,4]])`

Output:
`[[4,3],[2,1]]`

Assignment name : shift_cipher

Expected files : shift_cipher.py
Allowed functions : ord, chr

Write a function that shifts every alphabetic character by a given amount.

Lowercase letters must stay lowercase. Uppercase letters must stay uppercase.

The alphabet must wrap correctly.

Example:
- z shifted by 1 becomes a
- Z shifted by 1 becomes A

Non alphabetic characters must stay unchanged.

Example:
`shift_cipher("XYZ", 2)`

Output:
`"ZAB"`

LEVEL 5

Assignment name : compress

Expected files : compress.py
Allowed functions : len

Write a function that compresses repeated consecutive characters.

Each character must be followed by the number of times it appears consecutively.

Characters appearing once must still include the number 1.

You must return the compressed string.

Example:
`compress("aaabbcccc")`

Output:
`"a3b2c4"`

Assignment name : interval_merge

Expected files : interval_merge.py
Allowed functions : sorted

Write a function that merges overlapping intervals.

Two intervals overlap if one starts before the other ends.

The function must return a new list with merged intervals.

The intervals must remain sorted.

Example:
`[[1,3],[2,6],[8,10],[9,12]]`

Output:
`[[1,6],[8,12]]`

LEVEL 6

Assignment name : mini_interpreter

Expected files : mini_interpreter.py
Allowed functions : split, int

Write a function that interprets a sequence of instructions.

Supported instructions are:
- ADD x
- SUB x
- MUL x
- DIV x

The accumulator starts at 0.

Instructions are separated by spaces inside a single string.

The function must return the final accumulator value.

Example:
```
mini_interpreter("ADD 5 MUL 2 SUB 3")
```

Output:
7

Assignment name : maze_path

Expected files : maze_path.py
Allowed functions : len

Write a function that checks if a path exists inside a maze.

The maze is represented by a matrix containing:
- 0 for empty spaces
- 1 for walls

You receive:
- the grid
- a start position
- an end position

You may move:
- up
- down
- left
- right

Return True if a path exists. Return False otherwise.

Example:
```
maze_path(grid, (0,0), (2,2))
```